Tutorial – 3.1

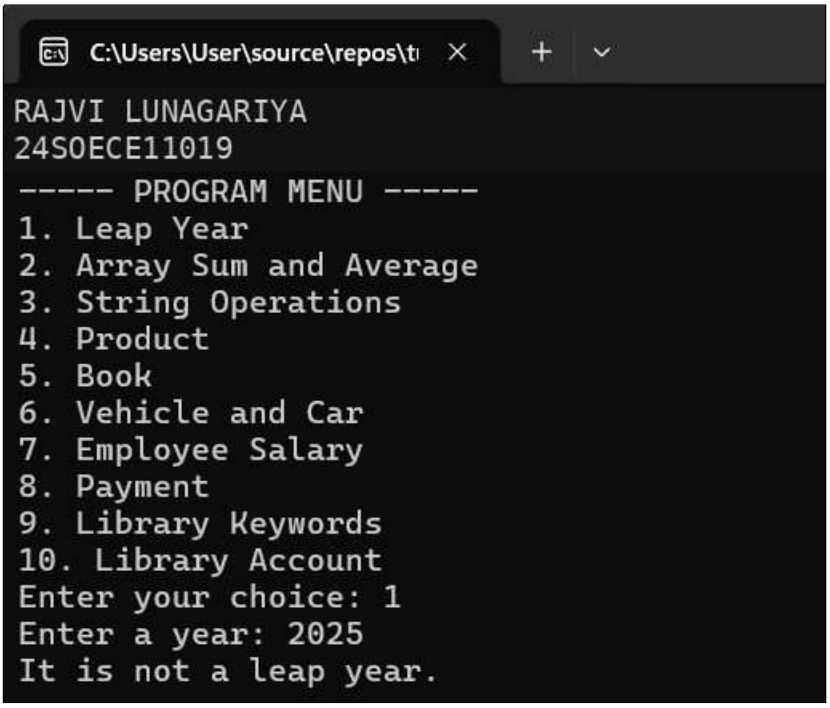
1. **Write a program to accept a year from the user and determine whether it is a leap year or not using a decision-making statement. Display the appropriate message.**

```
using System;

namespace tutorial3._1
{
   internal class Class1
   {
      public static void Run()
      {
         int year;

         Console.Write("Enter a year: ");
         year = Convert.ToInt32(Console.ReadLine());

         if (year % 400 == 0)
         {
            Console.WriteLine("It is a leap year.");
         }
         else if (year % 100 == 0)
         {
            Console.WriteLine("It is not a leap year.");
         }
         else if (year % 4 == 0)
         {
            Console.WriteLine("It is a leap year.");
         }
else
         {
            Console.WriteLine("It is not a leap year.");
         }
      }
   }
}
```



2. Write a program to accept N integer elements into an array and calculate the sum and average of all the elements using loops.

```
using System; using System.Diagnostics.Metrics;
using tutorial3._1; using static
System.Net.Mime.MediaTypeNames;

namespace tutorial3._1
{
   internal class Class2
   {
      public static void Run()
      {          int n;
int sum = 0;

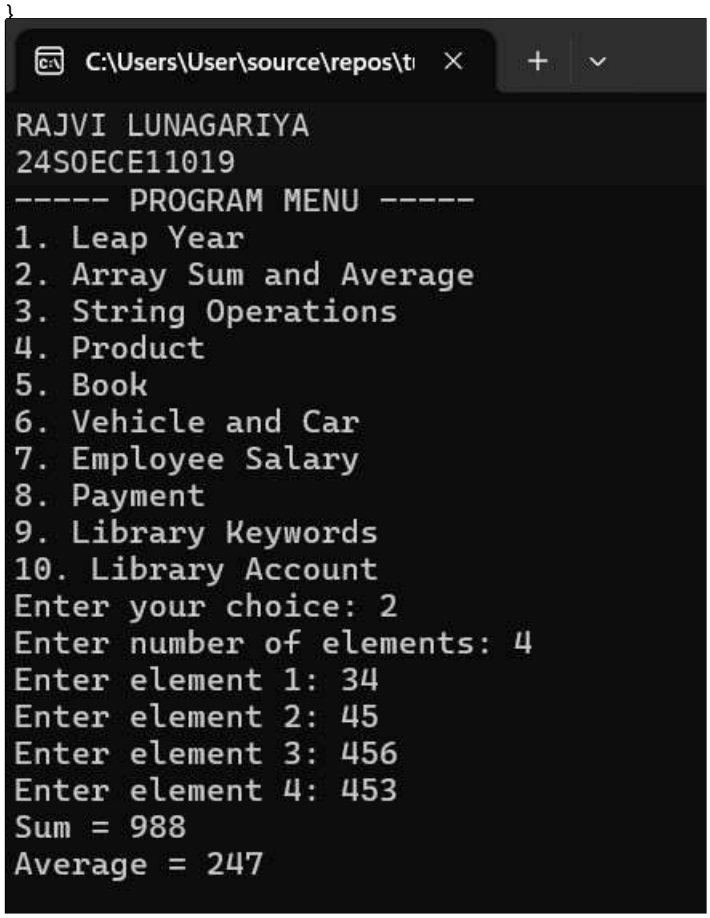
         Console.Write("Enter number of elements: ");
n = Convert.ToInt32(Console.ReadLine());

         int[] arr = new int[n];
          for (int i = 0; i < n;
i++)
         {
            Console.Write("Enter element " + (i + 1) + ": ");
            arr[i] = Convert.ToInt32(Console.ReadLine());
```

```
            sum = sum + arr[i];
        }

        double average = (double)sum / n;

        Console.WriteLine("Sum = " + sum);
        Console.WriteLine("Average = " + average);
    }
  }
}
```



3. Write a program to accept a sentence from the user and perform string operations such as converting it into uppercase, replacing spaces, trimming extra spaces, and finding its length.

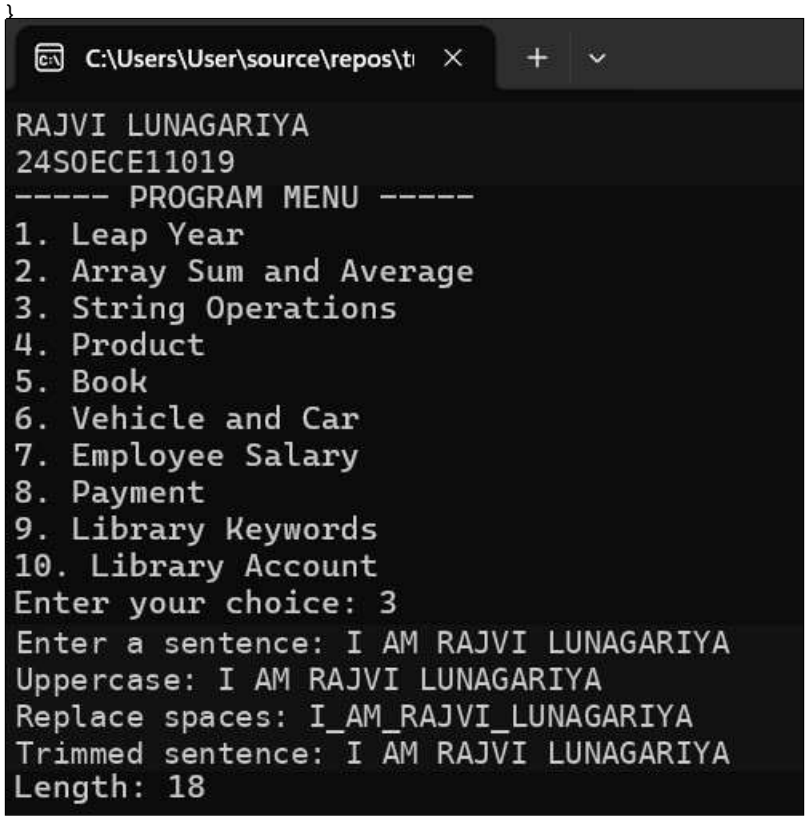
```
using System;

namespace tutorial3._1
```

```
{
    internal class Class3
    {
        public static void Run()
        {
            string sentence;

            Console.Write("Enter a sentence: ");
sentence = Console.ReadLine();

            Console.WriteLine("Uppercase: " + sentence.ToUpper());
            Console.WriteLine("Replace spaces: " + sentence.Replace(" ", "_"));
            Console.WriteLine("Trimmed sentence: " + sentence.Trim());
            Console.WriteLine("Length: " + sentence.Length);
        }
    }
}
```



4. Define a class Product with private data members for product code, product name, and price. Use a constructor and public methods to calculate discount and final price, and display the product details.

```
using System;
```

```
namespace tutorial3._1
{
   internal class Class4
   {
      private   int   productCode;
private   string   productName;
private double price;

      public Class4(int code, string name, double p)
      {
         productCode = code;
productName = name;          price
= p;
      }

      public double CalculateDiscount()
      {
         return price * 0.10;
      }

      public double CalculateFinalPrice()
      {
         return price - CalculateDiscount();
      }

      public void Display()
      {
         Console.WriteLine("Product Code: " + productCode);
         Console.WriteLine("Product Name: " + productName);
         Console.WriteLine("Price: " + price);
         Console.WriteLine("Discount: " + CalculateDiscount());
         Console.WriteLine("Final Price: " + CalculateFinalPrice());
      }

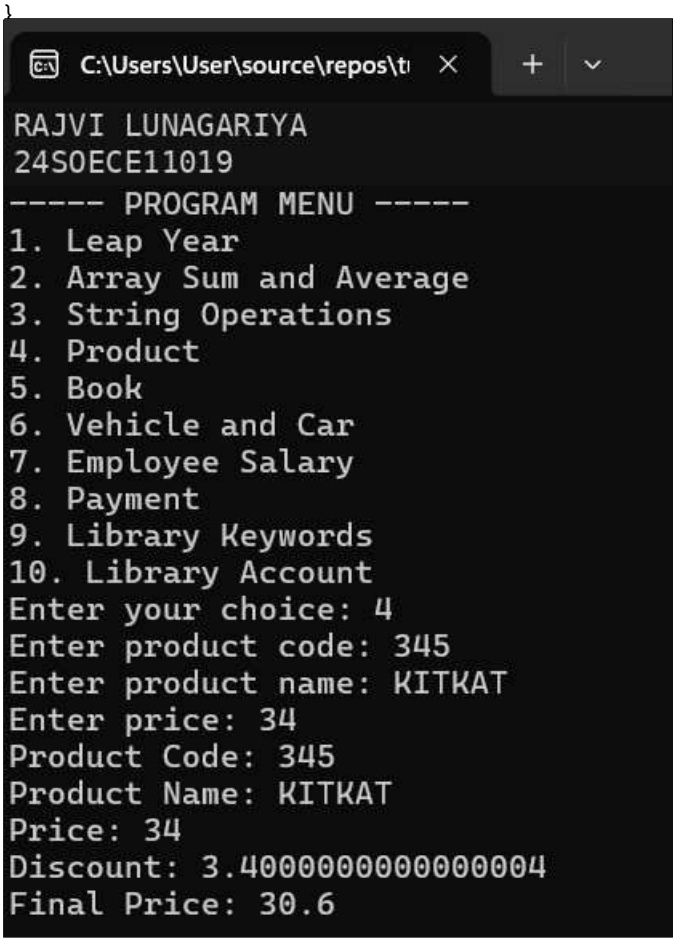
      public static void Run()
      {          int
code;          string
name;
double price;

         Console.Write("Enter product code: ");
code = Convert.ToInt32(Console.ReadLine());
```

```
            Console.Write("Enter product name: ");
name = Console.ReadLine();

            Console.Write("Enter price: ");              price =
Convert.ToDouble(Console.ReadLine());              Class4
product = new Class4(code, name, price);

            product.Display();
        }
    }
}
```



5. Define a class Book with book ID, title, author, and price using properties. Create multiple book objects and display their complete details using constructors and methods.

```
using System;

namespace tutorial3._1
```

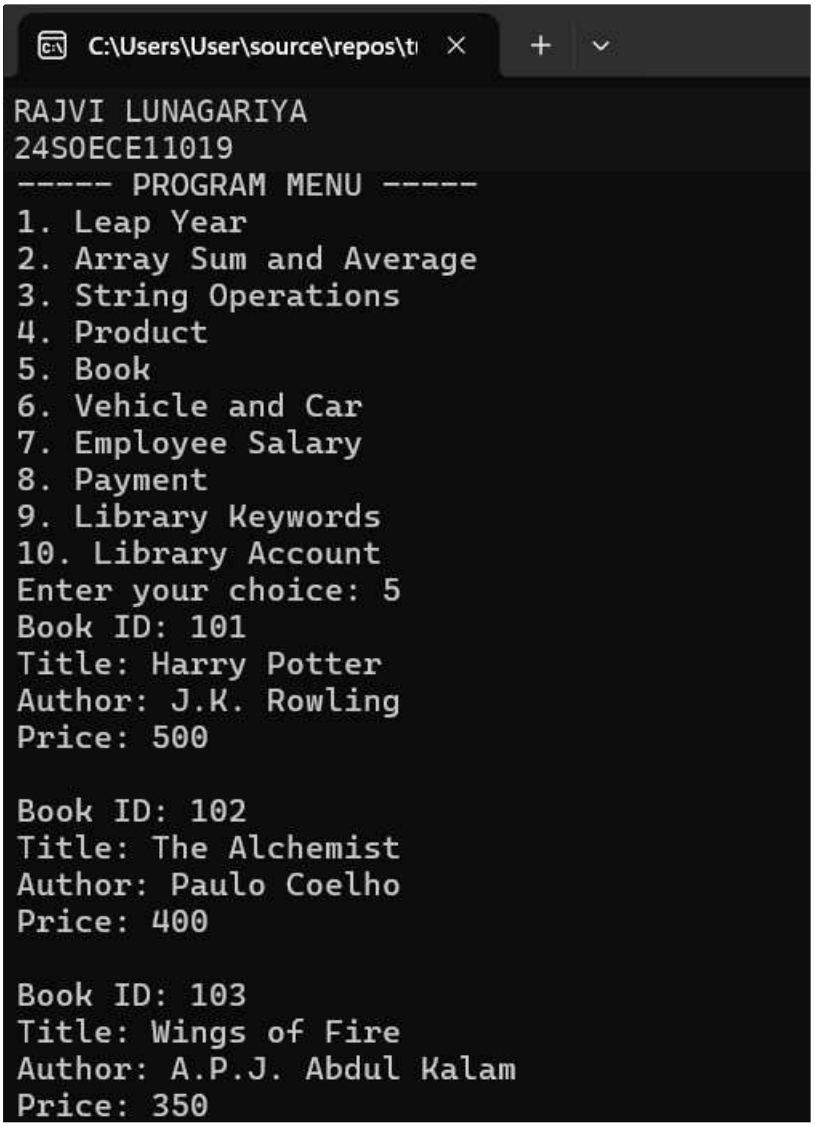
```
{
    internal class Class5
    {
        public int BookId { get; set; }
        public string Title { get; set; }
public string Author { get; set; }
public double Price { get; set; }

        public Class5(int id, string title, string author, double price)
        {
            BookId = id;
            Title = title;
            Author = author;
            Price = price;
        }

        public void Display()
        {
            Console.WriteLine("Book ID: " + BookId);
            Console.WriteLine("Title: " + Title);
            Console.WriteLine("Author: " + Author);
            Console.WriteLine("Price: " + Price);
            Console.WriteLine();
        }

        public static void Run()
        {
            Class5 book1 = new Class5(101, "Harry Potter", "J.K. Rowling", 500);
            Class5 book2 = new Class5(102, "The Alchemist", "Paulo Coelho", 400);
            Class5 book3 = new Class5(103, "Wings of Fire", "A.P.J. Abdul Kalam", 350);

            book1.Display();
book2.Display();            book3.Display();
        }
    }
}
```



6. Define a base class Vehicle containing vehicle number and model name, and derive a Car class from it. Use the base class constructor and derived class methods to accept and display complete car details.

```
using System;

namespace tutorial3._1
{
   internal class Class6
   {
      public string vehicleNumber;
public string modelName;        public
Class6(string number, string model)
```

```
        {
            vehicleNumber = number;
modelName = model;
        }
    }

    internal class Car : Class6
    {
        public string carType;

        public Car(string number, string model, string type)
            : base(number, model)
        {
            carType = type;
        }

        public void Display()
        {
            Console.WriteLine("Vehicle Number: " + vehicleNumber);
            Console.WriteLine("Model Name: " + modelName);
            Console.WriteLine("Car Type: " + carType);
        }

        public static void Run()
        {
            string number;
string model;           string
type;

            Console.Write("Enter vehicle number: ");
number = Console.ReadLine();

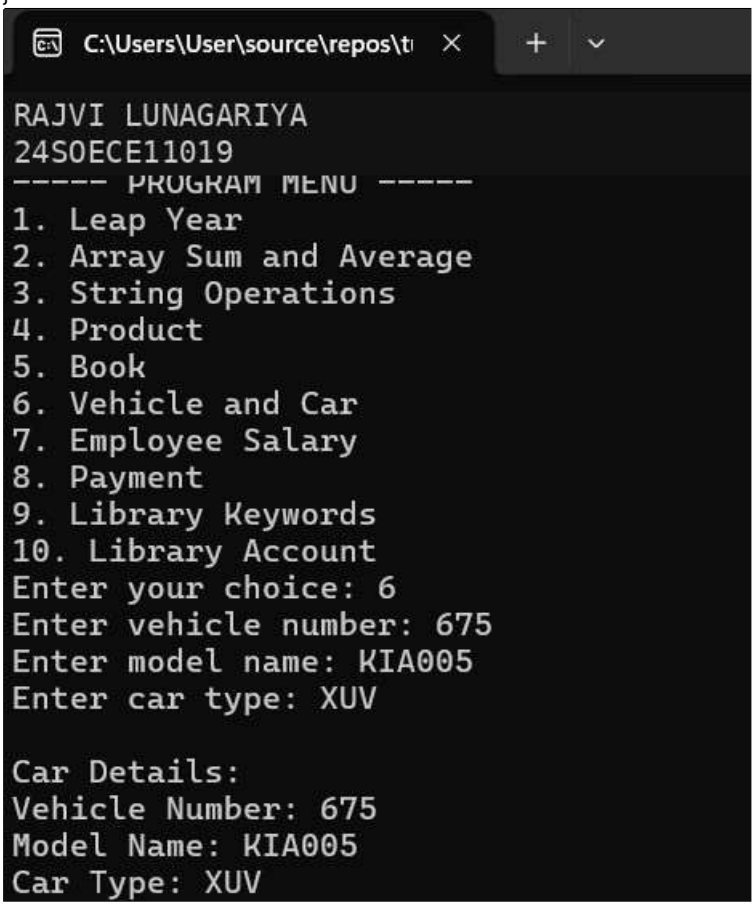
            Console.Write("Enter model name: ");
model = Console.ReadLine();

            Console.Write("Enter car type: ");
type = Console.ReadLine();

            Car car = new Car(number, model, type);

            Console.WriteLine();
            Console.WriteLine("Car Details:");
car.Display();
        }
    }
```

}



7. Define a base class Employee with a CalculateSalary() method and derive Manager and Developer classes from it. Override the CalculateSalary() method in each derived class and display their respective salaries.

```
using System;

namespace tutorial3._1
{
    internal class Class7
    {
        public virtual double CalculateSalary()
        {
            return 0;
        }
    }

    internal class Manager : Class7
    {
```

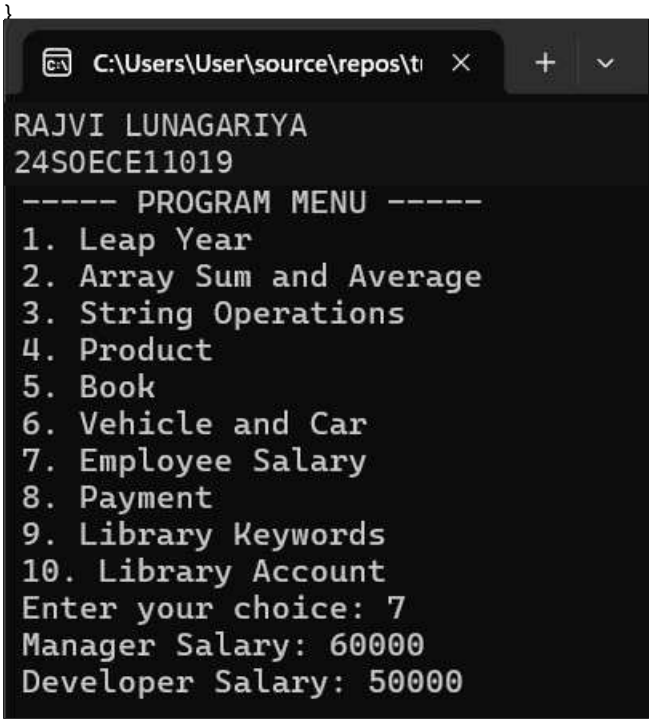
```
    public override double CalculateSalary()
    {
        return 60000;
    }
}

internal class Developer : Class7
{
    public override double CalculateSalary()
    {
        return 50000;
    }
}

internal class Salary
{
    public static void Run()
    {
        Manager manager = new Manager();
        Developer developer = new Developer();

        Console.WriteLine("Manager Salary: " + manager.CalculateSalary());
        Console.WriteLine("Developer Salary: " + developer.CalculateSalary());
    }
```

```
    }
}
```



8. Define an abstract class Payment with an abstract method to calculate the payment amount. Create derived classes for different payment types and implement the calculation using method overriding.

```
using System;

namespace tutorial3._1
{
    abstract class Class8
    {
        public abstract double CalculatePayment();
    }

    class CashPayment : Class8
    {
        public override double CalculatePayment()
        {
            return 5000;
        }
    }

    class CardPayment : Class8
```

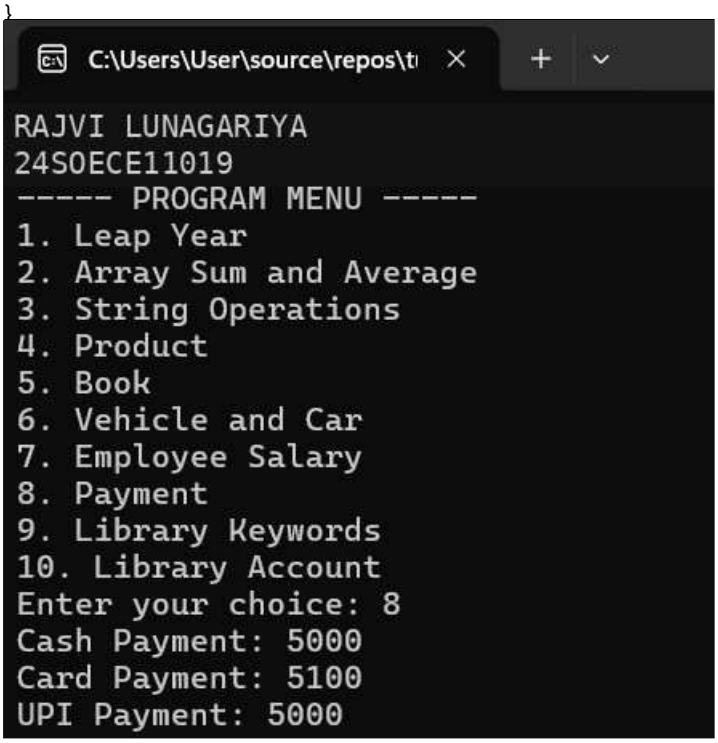
```
{
    public override double CalculatePayment()
    {
        return 5000 + 100;
    }
}

class UpiPayment : Class8
{
    public override double CalculatePayment()
    {
        return 5000;
    }
}

internal class Payment
{
    public static void Run()
    {
        CashPayment cash = new CashPayment();
        CardPayment card = new CardPayment();
        UpiPayment upi = new UpiPayment();

        Console.WriteLine("Cash Payment: " + cash.CalculatePayment());
        Console.WriteLine("Card Payment: " + card.CalculatePayment());
Console.WriteLine("UPI Payment: " + upi.CalculatePayment());
    }
```

```
    }
}
```



9. Write a program to demonstrate the use of static, this, base, and new keywords using suitable classes related to a library management system. Display the result of each operation.

```
using System;

namespace tutorial3._1
{
    class Library
    {
        public static string libraryName = "City Library";

        public string bookName;

        public Library(string bookName)
        {
            this.bookName = bookName;
        }

        public virtual void Display()
        {
            Console.WriteLine("Library Name: " + libraryName);
```

```
            Console.WriteLine("Book Name: " + this.bookName);
        }
    }

    class Book : Library
    {
        public string author;

        public Book(string bookName, string author)
            : base(bookName)
        {
            this.author = author;
        }

        public new void Display()
        {
            base.Display();
            Console.WriteLine("Author: " + this.author);
        }
    }

    internal class Class9
    {
        public static void Run()
        {
            Book book = new Book("The Alchemist", "Paulo Coelho");

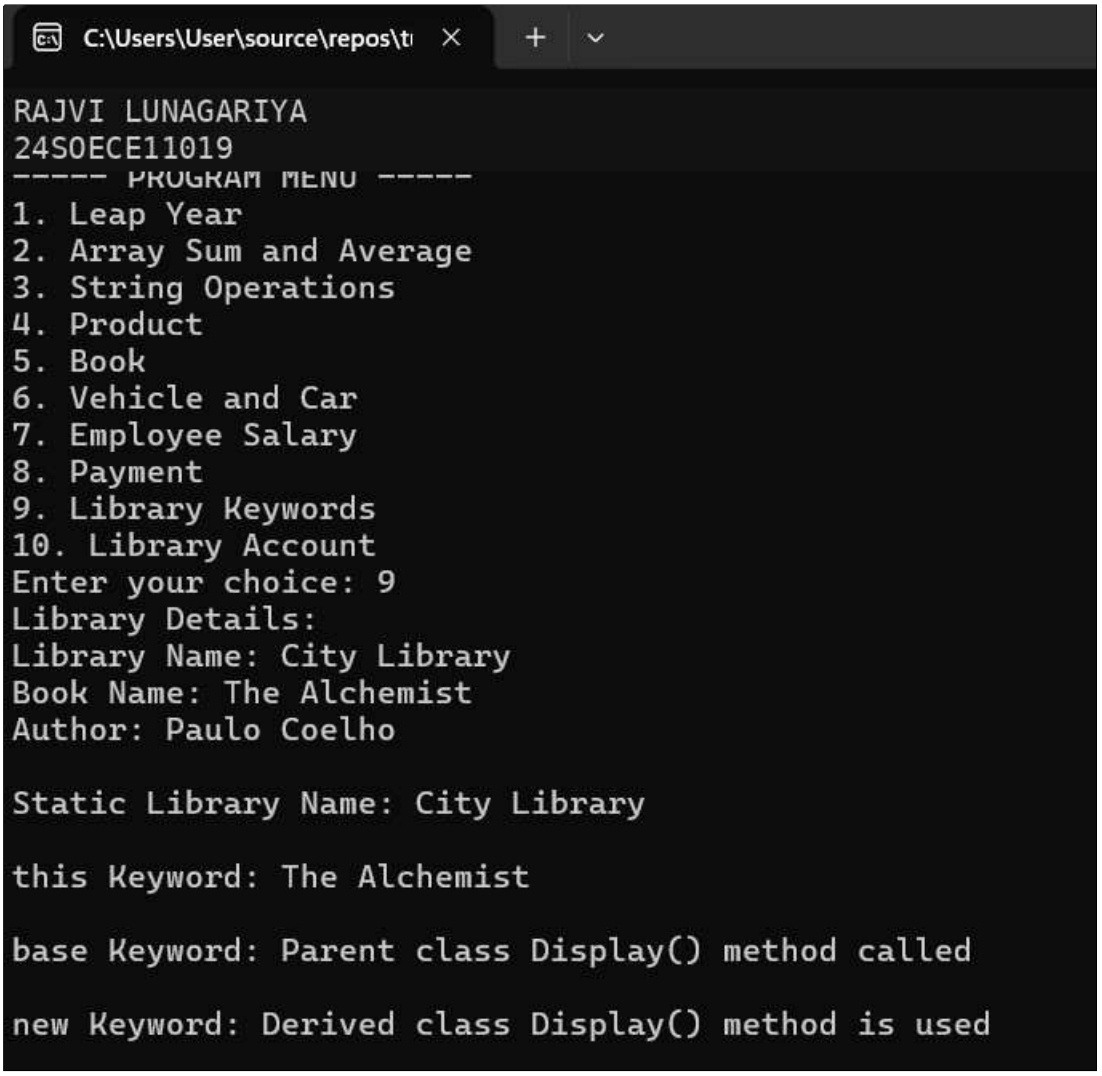
            Console.WriteLine("Library Details:");
book.Display();

            Console.WriteLine();
            Console.WriteLine("Static Library Name: " + Library.libraryName);

            Console.WriteLine();
            Console.WriteLine("this Keyword: " + book.bookName);

            Console.WriteLine();
            Console.WriteLine("base Keyword: Parent class Display() method called");

            Console.WriteLine();
            Console.WriteLine("new Keyword: Derived class Display() method is used");
        }
    }
}
```



10. Define a class LibraryAccount with a private number of issued books and public methods to issue, return, and display the number of books. Create multiple objects and demonstrate controlled access to the data.

```
using System;

namespace tutorial3._1
{
    internal class Class10
    {
        private int issuedBooks;

        public Class10()
```

```
        {
            issuedBooks = 0;
        }

        public void IssueBook()
        {
            issuedBooks++;
        }

        public void ReturnBook()
        {
            if (issuedBooks > 0)
            {
                issuedBooks--;
            }
        }

        public void DisplayBooks()
        {
            Console.WriteLine("Number of issued books: " + issuedBooks);
        }

        public static void Run()
        {
            Class10 account1 = new Class10();
            Class10 account2 = new Class10();

            account1.IssueBook();
account1.IssueBook();

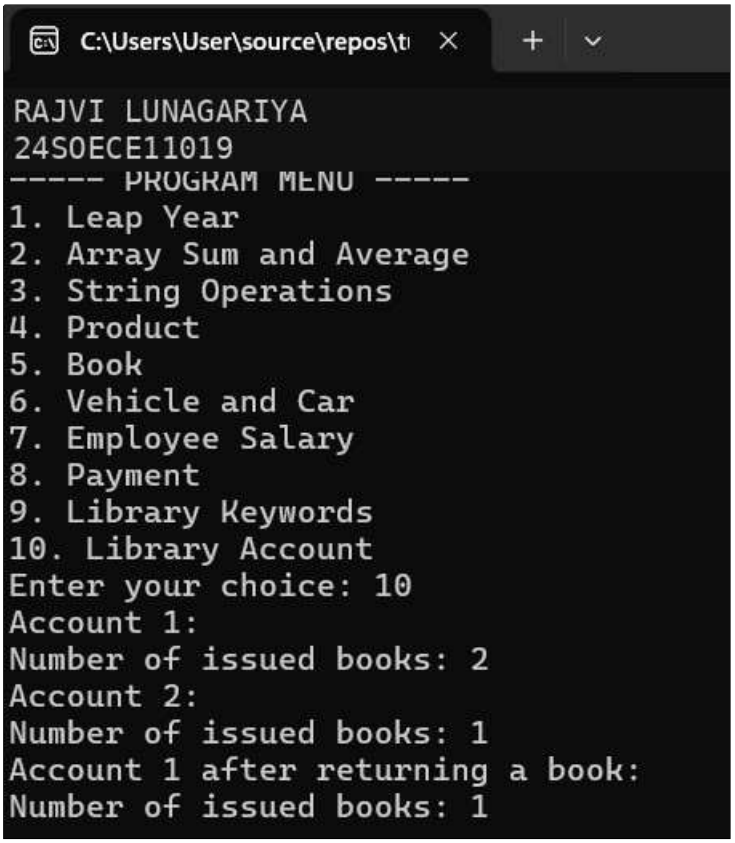
            account2.IssueBook();

            Console.WriteLine("Account 1:");
account1.DisplayBooks();

            Console.WriteLine("Account 2:");
account2.DisplayBooks();

            account1.ReturnBook();

            Console.WriteLine("Account 1 after returning a book:");
account1.DisplayBooks();
        }
    }
}
```


C:\Users\User\source\repos\t
RAJVI LUNAGARIYA
24SOECE11019
----- PROGRAM MENU -----
1. Leap Year
2. Array Sum and Average
3. String Operations
4. Product
5. Book
6. Vehicle and Car
7. Employee Salary
8. Payment
9. Library Keywords
10. Library Account
Enter your choice: 10
Account 1:
Number of issued books: 2
Account 2:
Number of issued books: 1
Account 1 after returning a book:
Number of issued books: 1